



UNIVERSIDADE FEDERAL DA FRONTEIRA SUL - *CAMPUS* CHAPECÓ
CURSO DE BACHAREL EM CIÊNCIA DA COMPUTAÇÃO
PROGRAMAÇÃO II
DOCENTE: GIANCARLO DONDONI SALTON

CAROLINE DE QUADROS PIAZZA (20230000690) E MAIQUELI EDUARDA DAMA
MINGOTI (20230004643)

TRABALHO INTEGRADOR:
ARQUITETURA GERAL

14 de julho de 2025

SUMÁRIO

1	Arquitetura	3
2	Backend	3
2.1	Configuração do Ambiente e Dependências	3
2.2	Controle de Versão	4
2.3	Configuração e Dependências	4
2.4	Estrutura de Pacotes do Backend	4
2.5	Arquitetura e Estrutura do Código	5
2.6	Modelo de Dados e Persistência	6
2.7	Segurança: Autenticação e Autorização	6
2.8	Funcionalidades de Negócio e Otimizações	6
2.9	Tratamento de Erros Centralizado	7
2.10	Anotações Utilizadas	7
2.10.1	Anotações do Spring e Injeção de Dependência	7
2.10.2	Anotações da API REST (Controllers)	7
2.10.3	Anotações do JPA (Entidades e Banco de Dados)	8
2.11	Visualização, Ordenação e Filtragem de Dados	8
3	Frontend	9
4	Database	10
4.1	Estrutura Principal e Herança	10
4.2	Relacionamentos e Integridade dos Dados	10
4.3	Validação e Testes com psql	10
5	Conclusão	11

1 Arquitetura

O sistema 2AG foi desenvolvido com uma arquitetura cliente-servidor, separando a lógica de negócios (backend) da interface do utilizador (frontend). Esta abordagem modular permite um desenvolvimento e manutenção mais fáceis.

Backend: Construído em Java 17 com o framework Spring Boot, é responsável por toda a lógica de negócio, gestão de dados e segurança. Utiliza o Spring Data JPA para interagir com a base de dados e o Maven para gestão de dependências.

Frontend: Desenvolvido com React e Vite, oferece uma interface de utilizador reativa e moderna. O npm é usado para gerir as dependências do frontend.

Banco de dados: Utiliza PostgreSQL para armazenar todos os dados da aplicação, desde informações dos utilizadores até registos clínicos detalhados.

Integração: A comunicação entre o frontend e o backend é feita através de uma API RESTful, com o backend a expor endpoints que o frontend consome para apresentar e manipular os dados.

2 Backend

O backend do sistema foi desenvolvido como uma API RESTful, utilizando o framework Spring Boot (versão 3.5.0) com a linguagem Java 17. O objetivo central foi criar uma infraestrutura robusta, segura e escalável para gerenciar toda a lógica de negócio e a persistência de dados da aplicação.

2.1 Configuração do Ambiente e Dependências

A configuração e as dependências do projeto são gerenciadas pelo Maven, com o arquivo `pom.xml` servindo como o descritor central. As dependências primárias do Spring Boot incluem:

- **spring-boot-starter-web:** Fornece um servidor Tomcat embutido e o suporte essencial para a criação de endpoints REST.
- **spring-boot-starter-data-jpa:** Integra o Hibernate para o mapeamento objeto-relacional (ORM) com o banco de dados PostgreSQL.
- **spring-boot-starter-security:** Módulo principal para a implementação da camada de segurança da aplicação.
- **jjwt:** Biblioteca utilizada para a criação, assinatura e validação dos JSON Web Tokens.

As configurações de ambiente são definidas no arquivo `application.yml`. Este arquivo especifica a porta do servidor (8080), as credenciais de conexão com o banco de dados PostgreSQL,



e a propriedade `spring.jpa.hibernate.ddl-auto: update`, que automatiza a atualização do esquema do banco de dados. Além disso, contém a chave secreta para a assinatura dos tokens JWT.

2.2 Controle de Versão

O desenvolvimento do projeto foi gerenciado através do sistema de controle de versão Git, com o repositório hospedado na plataforma GitHub. Esta abordagem permitiu o versionamento contínuo do código-fonte e a colaboração entre os desenvolvedores. O repositório está disponível em: <https://github.com/maiqueli-mingoti/2ag>

2.3 Configuração e Dependências

A fundação do projeto é definida pelo arquivo `pom.xml`, que gerencia todas as dependências através do Maven. As dependências cruciais incluem:

- **Spring Boot Starter Web:** Fornece um servidor web Tomcat embutido e as funcionalidades essenciais para a criação de APIs REST.
- **Spring Boot Starter Data JPA:** Facilita a interação com o banco de dados PostgreSQL, utilizando o Hibernate como provedor de ORM.
- **Spring Boot Starter Security:** Implementa a camada de segurança da aplicação.
- **JWT (Java JWT):** Biblioteca para a criação e validação dos JSON Web Tokens.
- **PostgreSQL Driver:** O driver JDBC que permite a comunicação entre a aplicação e o banco de dados.

A configuração do ambiente de execução é centralizada no arquivo `application.yml`. Nele, definimos a porta do servidor (8080), as credenciais de conexão com o banco de dados e a propriedade `spring.jpa.hibernate.ddl-auto: update`, que instrui o Hibernate a atualizar o esquema do banco a cada inicialização. Para a segurança, este arquivo também armazena a chave secreta utilizada para assinar e validar os tokens JWT.

2.4 Estrutura de Pacotes do Backend

A organização do código-fonte do backend foi modularizada em pacotes, cada um com uma responsabilidade única, seguindo as melhores práticas do Spring Boot para garantir a manutenibilidade e a clareza da arquitetura.

- **controller:** Contém as classes `@RestController` que definem os endpoints da API REST. Atuam como a camada de entrada, recebendo requisições HTTP e delegando o processamento para a camada de serviço. Exemplo: `PatientsController.java`.
- **dto:** Armazena os Data Transfer Objects. São classes ou records simples, como o `LoginDTO`, utilizados para transferir dados entre o cliente e o servidor, desacoplando a API do modelo de dados interno.
- **enums:** Agrupa os tipos enumerados da aplicação, como `ScaleType` e `AssignmentStatus`, garantindo a consistência e a segurança de tipos para valores constantes.
- **infra:** Destinado a componentes de infraestrutura e configurações transversais. Atualmente, contém a classe `ErrorHandler`, que centraliza o tratamento de exceções da aplicação.
- **model:** Contém as classes de entidade JPA, anotadas com `@Entity`, que representam as tabelas do banco de dados, como `Patient` e `Prescriber`.
- **repository:** Define as interfaces que estendem o `JpaRepository`. São responsáveis pela camada de acesso a dados, provendo métodos para operações de persistência. Exemplo: `PatientRepository.java`.
- **security:** Encapsula todas as classes relacionadas à segurança, incluindo a configuração do Spring Security (`SecurityConfigurations`), o filtro de validação de tokens (`SecurityFilter`) e o serviço de geração de JWT (`TokenService`).
- **service:** Contém a lógica de negócio da aplicação. Classes como o `PrescriberService` implementam as regras de negócio, como a geração de códigos únicos e validações, antes de interagir com a camada de repositório.

2.5 Arquitetura e Estrutura do Código

A aplicação foi estruturada seguindo o padrão arquitetural de três camadas, visando a separação de responsabilidades e a manutenibilidade do código:

- **Camada de Controller:** Define os endpoints da API (e.g., `/prescritor`, `/acompanhamento`) e é responsável por receber as requisições HTTP, validar e extrair dados, e invocar os métodos na camada de serviço.
- **Camada de Service:** É o núcleo da aplicação, onde reside a lógica de negócio. O `PrescriberService`, por exemplo, contém as regras para a criação de um novo prescritor, incluindo a validação de unicidade do registro profissional e a geração automática do código de vínculo (`professionalCode`).

- **Camada de Repository:** Composta por interfaces que estendem o `JpaRepository`, esta camada abstrai o acesso ao banco de dados, com métodos de busca customizados sendo gerados automaticamente pelo Spring Data JPA a partir da assinatura do método.

2.6 Modelo de Dados e Persistência

O modelo de dados foi implementado utilizando Entidades JPA. A estrutura utiliza herança, com uma classe abstrata `Users` definindo os atributos comuns. As classes `Patient` e `Prescriber` estendem `Users` e adicionam seus atributos específicos, utilizando a estratégia `InheritanceType.JOINED`. As relações entre as entidades são mapeadas com anotações como `@ManyToOne`, e para evitar problemas de recursão infinita na serialização de JSON, foram utilizadas as anotações `@JsonManagedReference` e `@JsonBackReference`.

2.7 Segurança: Autenticação e Autorização

A segurança da API é garantida pelo Spring Security, configurado para operar de forma "stateless" com tokens JWT.

- **Autenticação:** O fluxo se inicia no endpoint público `POST /auth/login`. O `AuthenticationManager` utiliza o `AuthorizationService` para buscar o usuário no banco pelo e-mail e validar a senha criptografada.
- **Geração de Token:** Após a autenticação, o `TokenService` gera um token JWT assinado, contendo "claims" como o ID, nome e perfis de autorização do usuário (`ROLE_ADMIN` para prescritores e `ROLE_USER` para pacientes).
- **Autorização:** Para endpoints protegidos, o cliente deve enviar o token JWT no cabeçalho `Authorization: Bearer <token>`. O `SecurityFilter` intercepta e valida o token, configurando o contexto de segurança para autorizar ou negar o acesso.

2.8 Funcionalidades de Negócio e Otimizações

Além do CRUD básico, o backend implementa lógicas mais complexas. O **fluxo de tarefas**, implementado pela entidade `AssignedScale`, permite que um prescritor designe um formulário a um paciente, cujo status é atualizado de `PENDENTE` para `CONCLUIDO` automaticamente após o preenchimento. Para otimizar a performance, foram criados **endpoints de agregação**. A rota `GET /dashboard/prescritor/{id}`, por exemplo, retorna um DTO com dados já consolidados. De forma similar, a rota `GET /pacientes/{id}/progresso` realiza a lógica de filtragem no backend, retornando dados prontos para a plotagem de gráficos.

2.9 Tratamento de Erros Centralizado

A classe `ExceptionHandler`, anotada com `@RestControllerAdvice`, intercepta exceções lançadas em qualquer parte da aplicação. Isso permite padronizar as respostas de erro. Uma `ValidationException` lançada por um serviço (ex: "e-mail já cadastrado") é capturada e transformada em uma resposta `400 Bad Request` com uma mensagem clara, fornecendo um feedback consistente para a API cliente.

2.10 Anotações Utilizadas

No desenvolvimento do backend, optou-se por não utilizar as anotações `@Getter` e `@Setter` da biblioteca Lombok. Embora pudessem reduzir a verbosidade do código, a escolha foi por escrever os métodos de acesso manualmente para manter um controle explícito sobre as propriedades de cada entidade. As anotações do framework Spring e do JPA foram a base para a construção da aplicação.

2.10.1 Anotações do Spring e Injeção de Dependência

- `@SpringBootApplication`: Anotação principal que combina `@Configuration`, `@ComponentScan` e `@EnableAutoConfiguration`, inicializando o contexto da aplicação e suas configurações automáticas.
- `@Service`, `@Repository`, `@Component`: Estereótipos que marcam as classes para serem gerenciadas pelo contêiner de injeção de dependência do Spring, definindo seus papéis na arquitetura (lógica de negócio, acesso a dados e componentes genéricos, respectivamente).
- `@Configuration` e `@Bean`: Utilizadas para definir configurações e expor objetos (Beans) para serem gerenciados pelo Spring, como o `PasswordEncoder` e o `SecurityFilterChain`.
- `@Value`: Injeta valores diretamente do arquivo de propriedades `application.yml`, utilizado para configurar a chave secreta do JWT no `TokenService`.

2.10.2 Anotações da API REST (Controllers)

- `@RestController`: Define uma classe como um controller REST, cujos métodos retornam dados que são automaticamente serializados para JSON.
- `@RequestMapping`: Mapeia o caminho base de URL para um controller inteiro.
- `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`: Anotações específicas para mapear os métodos HTTP para os métodos do controller.

- **@RequestBody** e **@PathVariable**: Utilizadas para extrair dados do corpo da requisição e da URL, respectivamente, convertendo-os em parâmetros para os métodos.
- **@CrossOrigin**: Configura a política de Cross-Origin Resource Sharing, permitindo que o frontend acesse a API a partir de uma origem diferente.

2.10.3 Anotações do JPA (*Entidades e Banco de Dados*)

- **@Entity**: Marca uma classe Java como uma entidade JPA, mapeando-a para uma tabela no banco de dados.
- **@Id** e **@GeneratedValue**: Definem a chave primária da entidade e a estratégia de geração de seu valor (no caso, `GenerationType.IDENTITY`).
- **@Column(unique = true)**: Aplica uma restrição de unicidade em nível de banco de dados para campos como e-mail e CPF.
- **@Table(uniqueConstraints = {...})**: Utilizada na entidade `Prescriber` para definir uma restrição de unicidade composta, abrangendo as colunas `registry_type` e `registry_number`.
- **@ManyToOne** e **@OneToMany**: Mapeiam os relacionamentos bidirecionais entre as entidades, como a associação entre `Patient` e `Prescriber`.
- **@JsonManagedReference** e **@JsonBackReference**: Anotações do Jackson utilizadas para gerenciar a serialização de relacionamentos bidirecionais, prevenindo a recursão infinita.
- **@MappedSuperclass**: Define que os campos de uma classe base, como `BaseAssessment`, devem ser mapeados nas entidades que a herdam, sem que a classe base se torne uma tabela.

2.11 Visualização, Ordenação e Filtragem de Dados

Para atender aos requisitos de visualização e manipulação de listas de dados, a API foi projetada para suportar funcionalidades de ordenação e filtragem através de parâmetros de consulta (query params) nas requisições GET.

- **Ordenação**: Em endpoints que retornam listas, como `GET /paciente`, o cliente pode especificar o campo para ordenação. Por exemplo, uma chamada para `/paciente?sort=name,asc` instrui o backend a retornar a lista de pacientes ordenada pelo nome em ordem ascendente. Esta funcionalidade é implementada no backend utilizando o objeto `Pageable` do Spring Data, que traduz os parâmetros de ordenação diretamente para a consulta SQL.

- **Filtragem:** A API permite a filtragem de dados em múltiplos contextos. Nos relatórios de progresso (/pacientes/{id}/progresso), é possível filtrar os dados por um atributo específico e por um período de tempo (ex: ?atributo=DOR&periodo=DIAS_30). Para listas de entidades, a filtragem textual pode ser implementada adicionando um parâmetro de busca, como /paciente?search=joão, que realizaria uma busca por nome no banco de dados.

Esta abordagem centraliza a lógica de consulta no backend, otimizando a performance e entregando ao frontend os dados já ordenados e filtrados, prontos para exibição.

3 Frontend

O frontend foi desenvolvido em JavaScript de maneira modular e intuitiva consumindo dados fornecidos pelo backend. Para construção do frontend foi utilizado como biblioteca principal o **React**. O **Vite** foi utilizado como ferramenta de build e servidor de desenvolvimento. Além disso a biblioteca **React Router DOM** responsável pela navegação (roteamento) do lado do cliente. Permite que a aplicação tenha múltiplos URLs (/login, /dashboard) sem necessitar de recarregar a página, criando uma experiência de navegação fluida.

A pasta **public** contém os recursos estáticos, como o `index.html` inicial, imagens e outros ficheiros que não necessitam de processamento.

Já o **src** o coração da aplicação, onde reside todo o código-fonte. A navegação é gerida no ficheiro `routes.jsx` usando o `react-router-dom`.

A aplicação utiliza principalmente os Hooks nativos do React para a gestão do estado local dos componentes:

- **useState:** Usado para armazenar dados que podem mudar ao longo do tempo e que devem acionar uma nova renderização da UI quando alterados. Por exemplo, para guardar os dados de um formulário ou a lista de pacientes recebida da API.
- **useEffect:** Utilizado para executar "efeitos secundários" (operações que não estão diretamente relacionadas com a renderização), como o carregamento de dados de uma API.

A metodologia de estilização adota uma abordagem dual. Primeiramente, um sistema de design (design system) é instituído através de arquivos de estilo globais, localizadas em `/src/styles`. Arquivos como `colors.css` e `button.css` utilizam variáveis CSS (`:root`) para garantir a consistência visual e facilitar a manutenção em toda a aplicação. Em segundo lugar, adota-se o encapsulamento de estilos através de arquivos CSS específicos para cada componente de página. Esta prática previne conflitos de especificidade e assegura que as regras de estilo de um componente não afetem outros de maneira não intencional.

4 Database

A persistência de dados do sistema é gerenciada pelo **PostgreSQL**, um sistema de gerenciamento de banco de dados relacional objeto. A estrutura do banco foi projetada de forma normalizada para garantir a integridade e a consistência dos dados. A comunicação entre a aplicação e o banco é abstraída pela camada de persistência do Spring Data JPA, com o Hibernate como provedor de ORM (Mapeamento Objeto-Relacional).

4.1 Estrutura Principal e Herança

A base do modelo de dados é a tabela `users`, que armazena os atributos comuns a todos os perfis de usuário, como nome, e-mail, CPF e senha. Para representar as especializações, foi adotada a estratégia de herança `InheritanceType.JOINED`. Com isso, as tabelas `patient` e `prescriber` contêm apenas os campos específicos de cada perfil, enquanto compartilham os dados comuns através de uma chave primária que também atua como chave estrangeira para a tabela `users`. Esta abordagem otimiza o armazenamento e mantém uma clara separação de conceitos.

4.2 Relacionamentos e Integridade dos Dados

Os relacionamentos entre as tabelas são o pilar da organização dos dados clínicos. A principal associação é entre `Prescritor` e `Paciente`, onde um prescritor pode estar associado a múltiplos pacientes (*One-to-Many*).

Todas as entidades de avaliação, como `anamnesis`, `follow_up`, `sleep_log` e as diversas escalas clínicas, estão diretamente relacionadas à tabela `patient` através de uma chave estrangeira, garantindo que cada registro de acompanhamento seja vinculado a um único paciente. Uma exceção é a entidade `mental_state_exam`, que se relaciona diretamente com uma `appointment` (consulta), tratando-a como um artefato daquele evento clínico específico.

A integridade dos dados é reforçada por mecanismos do banco de dados. As chaves primárias são do tipo `bigserial`, que utilizam sequências para gerar valores únicos e auto-incrementais. Além disso, foram aplicadas restrições `UNIQUE` em colunas críticas como `email` e `cpf`, e uma restrição de unicidade composta na tabela `prescriber` para a combinação das colunas `registry_type` e `registry_number`.

4.3 Validação e Testes com psql

Antes que a aplicação pudesse ser executada pela primeira vez, foi necessário preparar o ambiente do PostgreSQL. O Hibernate, configurado com a propriedade `ddl-auto`, é capaz de gerenciar o

esquema (criar e atualizar tabelas), mas não pode criar o banco de dados ou o usuário de conexão a partir do zero.

Por isso, utilizando a interface de linha de comando **psql**, foram executados os comandos manuais para a criação da base de dados com `CREATE DATABASE doisag` e do usuário da aplicação com `CREATE USER admindoisag`. Em seguida, foram concedidos todos os **privilégios** (`GRANT ALL PRIVILEGES`) sobre o banco `doisag` para este usuário, garantindo que a aplicação teria as permissões necessárias para ler, escrever, criar tabelas e estabelecer as restrições de chaves estrangeiras.

Após a configuração inicial e com o backend em execução, a ferramenta `psql` continuou a ser utilizada para validação direta. Após a execução de operações de escrita através dos endpoints da API (ex: `POST` para criar um novo paciente), consultas `SELECT` eram executadas para inspecionar o estado das tabelas. Este método permitiu verificar em tempo real se os dados foram persistidos corretamente, se os relacionamentos foram estabelecidos e se a lógica da camada de serviço estava funcionando como o esperado.

5 Conclusão

A estrutura do sistema 2AG foi pensada usando tecnologias atuais para resolver um problema real. A escolha de separar o sistema em duas partes principais, a lógica de funcionamento (backend com `Spring Boot`) e a parte visual (frontend com `React`), foi acertada e deixou tudo mais organizado. A segurança, a forma como o sistema foi construído em camadas e a organização dos dados criaram uma base sólida. Por isso, o sistema é seguro, fácil de manter e preparado para futuras expansões. O projeto alcançou os seus objetivos, resultando numa plataforma de gestão funcional e confiável.

O desenvolvimento do projeto foi gerenciado por meio de um repositório de controle de versão, hospedado na plataforma `GitHub`, o que permitiu o versionamento e a colaboração contínua no código-fonte. Disponível em: <https://github.com/maiqueli-mingoti/2ag>